МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №10

"Пакування, контейнеризація та CI/CD"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Варіант 1. Система обліку студентів. Мета роботи: освоїти повний життєвий цикл розгортання та пакування Python-застосунків (sdist, wheel через setuptools), налаштування конфігурації середовища (.env), контейнеризацію за допомогою Docker з використанням volume для постійного збереження даних SQLite, а також проектування пайплайну автоматичної перевірки коду, тестів і збирання образів у GitHub Actions.

Теоретичні відомості:

Дистрибутиви Python поширюються у форматах sdist (вихідний код) та wheel (скомпільований та підготовлений архів). Для забезпечення відтворюваності розгортання застосовується контейнеризація в Docker, де застосунок ізолюється разом з усіма залежностями. Стан бази даних у контейнері зберігається за допомогою Docker Volumes, які переживають перезапуск та заміну контейнерів. Конвеєри безперервної інтеграції (CI/CD на базі GitHub Actions) автоматизують запуск статичних аналізаторів, тестів, перевірку стилю та валідацію збирання релізу на кожному коміті.

Виконання роботи

1. Варіант 1: постановка задачі, предметна область та конфігурація Docker

```
FROM python:3.11-slim AS builder
WORKDIR /build
COPY pyproject.toml README.md ./
COPY src/ ./src/
RUN python -m pip install --no-cache-dir build \
    && python -m build --wheel --outdir /wheels

FROM python:3.11-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    DATABASE_URL=sqlite:////app/data/students.sqlite \
    ENVIRONMENT=production \
    LOG_LEVEL=INFO
WORKDIR /app
COPY --from=builder /wheels/*.whl /wheels/
RUN python -m pip install --no-cache-dir /wheels/*.whl \
    && rm -rf /wheels \
    && groupadd --gid 10001 appuser \
    && useradd --uid 10001 --gid appuser --create-home appuser \
    && mkdir -p /app/data \
    && chown appuser:appuser /app/data
USER appuser
VOLUME ["/app/data"]
EXPOSE 8000
STOPSIGNAL SIGTERM
HEALTHCHECK --interval=10s --timeout=3s --start-period=10s --retries=3 \
    CMD python -c "import json, urllib.request; response = urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2); assert response.status == 200; assert json.load(response)['status'] == 'ok'"
CMD ["python", "-m", "uvicorn", "student_manager.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

2. Автоматизований пайплайн перевірки якості у GitHub Actions (.github/workflows/ci.yml)

```yaml
name: CI

"on":
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  verify:
    runs-on: ubuntu-latest
    timeout-minutes: 25
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12"]
    defaults:
      run:
        shell: bash
    env:
      PYTHONPATH: ""
      MYPYPATH: ""
      ENVIRONMENT: test
      LOG_LEVEL: INFO
      CONTAINER_NAME: student-manager-${{ github.run_id }}-${{ github.run_attempt }}
      VOLUME_NAME: student-manager-data-${{ github.run_id }}-${{ github.run_attempt }}
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
```

```yaml
    - name: Install development dependencies
      run: |
        python -m pip install --upgrade pip
        python -m pip install -e '.[dev]' build
    - name: Ruff lint
      run: python -m ruff check src tests lab6 lab9
    - name: Ruff format
      run: python -m ruff format --check src tests lab6 lab9
    - name: Type checking
      run: python -m mypy src
    - name: Tests with coverage
      run: python -m pytest tests --cov=student_manager --cov-branch --cov-report=term-missing --cov-fail-under=80
    - name: Build wheel and sdist
      run: |
        python -m build
        python - <<'PY'
        from pathlib import Path
        assert len(list(Path('dist').glob('*.whl'))) == 1
        assert len(list(Path('dist').glob('*.tar.gz'))) == 1
        PY
    - name: Prepare live API smoke test
      run: |
        cat > "$RUNNER_TEMP/api_smoke.py" <<'PY'
        import json
        import os
        import time
        import urllib.error
        import urllib.request
        from pathlib import Path

        base = os.environ['SMOKE_BASE_URL']
        state = Path(os.environ['SMOKE_STATE'])

        def request(path, payload=None):
            data = None if payload is None else json.dumps(payload).encode()
```

```
            req = urllib.request.Request(
                base + path, data=data,
                headers={'Content-Type': 'application/json'},
            )
            with urllib.request.urlopen(req, timeout=3) as response:
                assert response.status == (200 if data is None else 201)
                return json.load(response)

        for attempt in range(60):
            try:
                health = request('/health')
                break
            except (OSError, urllib.error.URLError):
                if attempt == 59:
                    raise
                time.sleep(1)
        assert health['status'] == 'ok', health
        assert health['environment'] == os.environ['ENVIRONMENT'], health
        schema = request('/openapi.json')
        assert '/students' in schema['paths']
        if os.environ.get('SMOKE_MODE', 'write') == 'write':
            group = request('/groups', {'name': 'CI group'})
            student = request('/students', {
                'first_name': 'CI', 'last_name': 'Student',
                'group_id': group['id'], 'average_grade': 91.5,
            })
            state.write_text(json.dumps(student), encoding='utf-8')
        expected = json.loads(state.read_text(encoding='utf-8'))
        actual = request('/students/' + str(expected['student_id']))
        assert actual == expected, (actual, expected)
        members = request('/groups/' + str(expected['group_id']) + '/students')
        assert any(item == expected for item in members), members
        stats = request('/groups/' + str(expected['group_id']) + '/statistics')
        assert stats['count'] == 1 and stats['average'] == 91.5, stats
        print('Live health, OpenAPI, student create/read and group statistics verified')
```

```
          PY
      - name: Install and smoke-test packaged application
        run: |
          python -m venv "$RUNNER_TEMP/package-env"
          py="$RUNNER_TEMP/package-env/bin/python"
          "$py" -m pip install dist/*.whl
          "$py" -m pip check
          cd "$RUNNER_TEMP"
          "$py" - <<'PY'
          from pathlib import Path
          import sys
          import student_manager
          assert Path(student_manager.__file__).is_relative_to(Path(sys.prefix)), student_manager.__file__
          PY
          "$RUNNER_TEMP/package-env/bin/student-manager" --demo
          export DATABASE_URL="sqlite:///$RUNNER_TEMP/package.sqlite"
          export SMOKE_BASE_URL=http://127.0.0.1:8765
          export SMOKE_STATE="$RUNNER_TEMP/package-state.json"
          "$py" -m uvicorn student_manager.api:app --host 127.0.0.1 --port 8765 > "$RUNNER_TEMP/package-api.log" 2>&1 &
          api_pid=$!
          trap 'kill -TERM "$api_pid" 2>/dev/null || true; cat "$RUNNER_TEMP/package-api.log"' EXIT
          "$py" "$RUNNER_TEMP/api_smoke.py"
          kill -TERM "$api_pid"
          wait "$api_pid"
          trap - EXIT
          cat "$RUNNER_TEMP/package-api.log"
      - name: Upload distributions
        if: matrix.python-version == '3.11'
        uses: actions/upload-artifact@v4
        with:
          name: distributions
          path: dist/*
          if-no-files-found: error
      - name: Build Docker image
        if: matrix.python-version == '3.11'
```

```
        run: docker build --tag student-manager:ci .
      - name: Run container with writable persistent volume
        if: matrix.python-version == '3.11'
        env:
          ENVIRONMENT: production
        run: |
          docker volume create "$VOLUME_NAME"
          docker run --detach --name "$CONTAINER_NAME" \
            --publish 127.0.0.1:8766:8000 \
            --mount "type=volume,source=$VOLUME_NAME,target=/app/data" \
            --env DATABASE_URL=sqlite:////app/data/students.sqlite \
            --env ENVIRONMENT --env LOG_LEVEL student-manager:ci
          export SMOKE_BASE_URL=http://127.0.0.1:8766
          export SMOKE_STATE="$RUNNER_TEMP/container-state.json"
          python "$RUNNER_TEMP/api_smoke.py"
          docker exec "$CONTAINER_NAME" python -c "import os; from pathlib import Path; assert os.getuid() != 0; p = Path('/app/data/students.sqlite'); assert p.is_file(); assert os.access(p.parent, os.W_OK); assert os.access(p, os.W_OK)"
          for attempt in $(seq 1 30); do
            health=$(docker inspect --format '{{.State.Health.Status}}' "$CONTAINER_NAME")
            if [ "$health" = healthy ]; then break; fi
            sleep 1
          done
          test "$health" = healthy
          docker stop --time 15 "$CONTAINER_NAME"
          test "$(docker inspect --format '{{.State.ExitCode}}' "$CONTAINER_NAME")" = 0
          docker rm "$CONTAINER_NAME"
          docker run --detach --name "$CONTAINER_NAME" \
            --publish 127.0.0.1:8766:8000 \
            --mount "type=volume,source=$VOLUME_NAME,target=/app/data" \
            --env DATABASE_URL=sqlite:////app/data/students.sqlite \
            --env ENVIRONMENT --env LOG_LEVEL student-manager:ci
          SMOKE_MODE=read python "$RUNNER_TEMP/api_smoke.py"
          docker stop --time 15 "$CONTAINER_NAME"
          test "$(docker inspect --format '{{.State.ExitCode}}' "$CONTAINER_NAME")" = 0
      - name: Container logs and cleanup
        if: always() && matrix.python-version == '3.11'
```

```
        run: |
          if docker container inspect "$CONTAINER_NAME" > /dev/null 2>&1; then
            docker logs "$CONTAINER_NAME"
            docker rm --force "$CONTAINER_NAME"
          fi
          if docker volume inspect "$VOLUME_NAME" > /dev/null 2>&1; then
            docker volume rm "$VOLUME_NAME"
          fi
```

3. Перевірка якості коду, статичного аналізу та запуск повного набору тестів

```
PS C:\Users\ponti> pytest tests -q && ruff check src tests && mypy src
522 passed, 1 warning, 63 subtests passed in 106.92s
All checks passed!
Success: no issues found in 38 source files

======================== CI GATES READY ========================
PS C:\Users\ponti> _
```

4. Відповіді на контрольні запитання

Питання: Що таке Python package?

Відповідь: Python package (пакет) - це спосіб організації коду у спільний простір імен за допомогою каталогу, що зазвичай містить файл __init__.py або оформлений як namespace package. Він дозволяє логічно групувати пов'язані модулі та повторно використовувати їх в інших проєктах.

Питання: Що таке distribution package?

Відповідь: Distribution package (пакет розповсюдження) - це зібраний версіонований архів або дистрибутивний файл (wheel або sdist), призначений для публікації в індексах на кшталт PyPI та інсталяції через pip. Він містить вихідні або скомпільовані файли разом з метаданими та переліком залежностей проєкту.

Питання: Що таке wheel?

Відповідь: Wheel - це стандартний бінарний формат дистрибуції Python-пакетів (PEP 427), що містить готові до використання файли та не вимагає етапу збирання чи виконання setup.py під час інсталяції. Завдяки цьому встановлення пакетів через pip відбувається значно швидше та надійніше.

Питання: Яке розширення має wheel?

Відповідь: Файли формату wheel завжди мають стандартне розширення .whl. У назві файлу вказуються назва пакета, версія, сумісність з інтерпретатором Python, ABI та цільова платформа.

Питання: Що таке sdist?

Відповідь: Sdist (source distribution) - це архів із вихідним кодом проєкту, метаданими та конфігурацією збирання, який зазвичай формується у форматі .tar.gz. Під час встановлення sdist менеджер pip спочатку збирає з нього wheel на машині користувача, а потім виконує інсталяцію.

Питання: Для чого використовується pyproject.toml?

Відповідь: Файл pyproject.toml є сучасним стандартом конфігурації Python-проєктів (PEP 517, PEP 518, PEP 621), що визначає інструменти збирання, метадані та залежності пакета. Також він слугує єдиним файлом налаштувань для лінтерів, форматерів і тестів, таких як ruff, mypy та pytest.

Питання: Що містить [build-system]?
Відповідь: Секція [build-system] містить перелік пакетів, необхідних для збирання проєкту (параметр requires), а також назву бекенду збирання (build-backend, наприклад setuptools.build_meta чи hatchling.build). Вона забезпечує незалежне й відтворюване складання дистрибутиву в ізольованому середовищі.

Питання: Що містить [project]?
Відповідь: Секція [project] містить стандартизовані декларативні метадані пакета: назву, версію, короткий опис, авторів, версію Python, ліцензію, залежності (dependencies), опціональні dev-залежності та консольні точки входу (scripts).

Питання: Як створити wheel?
Відповідь: Для створення wheel використовується команда 'python -m build --wheel' (або загальна команда 'python -m build'). Сформований бінарний пакет із розширенням .whl автоматично розміщується в каталозі dist/.

Питання: Як створити sdist?
Відповідь: Для створення sdist запускають команду 'python -m build --sdist' (або 'python -m build'). Утиліта створює стиснутий архів вихідного коду з розширенням .tar.gz у папці dist/.

Питання: Для чого використовується python -m build?
Відповідь: Утиліта 'python -m build' викликає стандартний збирач пакетів (PEP 517/518 build frontend), який створює чисте ізольоване віртуальне середовище, встановлює необхідний build-backend і генерує готові артефакти wheel та sdist у директорії dist/.

Питання: Що таке semantic versioning?
Відповідь: Semantic versioning (семантичне версіонування) - це стандартизована схема нумерації версій програмного забезпечення у форматі MAJOR.MINOR.PATCH. Вона формалізує зміну версій відповідно до сумісності публічного API та масштабу внесених змін.

Питання: Що означає MAJOR?
Відповідь: Номер MAJOR змінюється при внесенні несумісних (breaking) змін у публічний API проєкту, які вимагають модифікації коду клієнтських програм. Зміна мажорної версії сигналізує про втрату зворотної сумісності.

Питання: Що означає MINOR?
Відповідь: Номер MINOR збільшується при додаванні нової функціональності, яка є повністю зворотно сумісною з попередніми випусками в межах поточної мажорної версії.

Питання: Що означає PATCH?
Відповідь: Номер PATCH збільшується при випуску зворотно сумісних виправлень виявлених дефектів і помилок, які не додають нових можливостей і не ламають існуючий API.

Питання: Що таке environment variable?
Відповідь: Environment variable (змінна середовища) - це динамічна змінна на рівні операційної системи, значення якої зчитується процесом під час його виконання. Вона дозволяє конфігурувати поведінку програми ззовні без редагування вихідного коду.

Питання: Навіщо configuration виносити з коду?
Відповідь: Винесення конфігурації дозволяє запускати той самий код у development, test і production з різними параметрами без зміни source code. Змінні середовища відділяють

налаштування від коду, але самі по собі не гарантують захист secrets: потрібні обмеження доступу та відсутність секретів у логах і Git.

Питання: Що таке secret?

Відповідь: Secret (секрет) - це будь-які конфіденційні дані автентифікації або безпеки, такі як паролі, токени API, приватні ключі шифрування та облікові дані баз даних. Несанкціонований доступ до секретів несе загрозу безпеці всієї системи.

Питання: Наведіть приклади secrets.

Відповідь: Прикладами секретів є паролі до баз даних, приватні API-токени (GitHub PAT, OpenAI key), токени публікації в PyPI, секретні ключі сесій JWT та закриті ключі SSL/SSH.

Питання: Чому secrets не можна commit у Git?

Відповідь: Секрети не можна комітити в Git, оскільки вся історія репозиторію зберігається назавжди й стає доступною всім учасникам та автоматичним парсерам при публікації. Навіть після видалення файлу наступним комітом скомпрометований секрет залишається в історії Git.

Питання: Для чого використовується .env?

Відповідь: Файл .env зберігає локальні значення конфігурації у форматі KEY=value. Python і Docker не читають його автоматично: потрібен явний завантажувач, налаштування env_file у pydantic-settings або docker run --env-file .env. Pydantic-settings може читати значення без зміни os.environ.

Питання: Чому .env має бути у .gitignore?

Відповідь: Файл .env може містити паролі й локальні налаштування, тому .gitignore допомагає не додати його випадково у Git. Ігнорування не прибирає вже відстежуваний файл та старі commits; при витоку треба відкликати або замінити secret і окремо очистити відстеження та історію за потреби.

Питання: Для чого потрібний .env.example?

Відповідь: Файл .env.example слугує публічним шаблоном і документацією, де наведено повний список необхідних змінних оточення із пустими або безпечними тестовими значеннями без справжніх секретів. Розробники копіюють його у власний .env для швидкого старту проєкту.

Питання: Що таке Docker?

Відповідь: Docker - це платформа для збирання образів і запуску ізольованих контейнерів із застосунком та його залежностями. Вона робить середовище відтворюванішим, але сумісність архітектури CPU, ядра, мережі, volumes і зовнішніх сервісів усе одно потрібно перевіряти.

Питання: Що таке Dockerfile?

Відповідь: Dockerfile - це текстовий конфігураційний файл, що містить покрокові інструкції (FROM, WORKDIR, COPY, RUN, CMD) для автоматичного збирання Docker-образу.

Питання: Що таке Docker image?

Відповідь: Docker image (образ) - це статичний, незмінний шаблон файлової системи та налаштувань, сформований з декількох шарів тільки для читання. Він містить операційне середовище, код застосунку, залежності та вказівки щодо запуску.

Питання: Що таке Docker container?

Відповідь: Docker container (контейнер) - це ізольований процес виконання, запущений на основі визначеного Docker-образу, який використовує спільне ядро ОС хоста. Він має власну ізольовану файлову систему, мережу та простір процесів.

Питання: Чим image відрізняється від container?

Відповідь: Docker image є пасивним незмінним шаблоном (аналогом класу або інсталяційного пакета), тоді як container є активним працюючим екземпляром цього образу з власним шаром запису та станом виконання.

Питання: Для чого використовується docker build?

Відповідь: Команда docker build зчитує вказаний Dockerfile та контекст директорії і покроково виконує інструкції, формуючи готовий незмінний Docker-образ із призначеним тегом.

Питання: Для чого використовується docker run?

Відповідь: Команда docker run створює новий контейнер із вказаного образу і запускає його виконання, дозволяючи прокидати мережеві порти, монтувати томи та передавати змінні оточення.

Питання: Що робить EXPOSE?

Відповідь: Інструкція EXPOSE у Dockerfile документує мережеві порти, які контейнер слухатиме під час роботи. Вона має інформаційний характер і фактично не публікує порти на хості без використання прапорця -p у команді docker run.

Питання: Що робить CMD?

Відповідь: Інструкція CMD задає команду за замовчуванням та її аргументи, які виконуються при запуску контейнера. Значення CMD може бути легко перевизначене користувачем шляхом передачі власних аргументів команді docker run.

Питання: Для чого потрібний WORKDIR?

Відповідь: Інструкція WORKDIR визначає робочу директорію всередині контейнера для виконання всіх наступних інструкцій Dockerfile (RUN, CMD, COPY). Якщо зазначений каталог відсутній, Docker автоматично створить його.

Питання: Що робить COPY?

Відповідь: Інструкція COPY копіює визначені файли та каталоги з контексту збирання на хост-системі у файлову систему створюваного Docker-образу.

Питання: Для чого використовується .dockerignore?

Відповідь: Файл .dockerignore запобігає копіюванню непотрібних і конфіденційних файлів (наприклад, .git, .env, .venv, __pycache__) із хоста в контекст збирання Docker. Це прискорює створення образу, зменшує його розмір і блокує витік секретів у фінальні шари.

Питання: Що таке Docker volume?

Відповідь: Docker volume (том) - це виділений механізм збереження персистентних даних, керований Docker поза межами файлової системи контейнера. Дані у volume зберігаються незалежно від життєвого циклу контейнера і не втрачаються після його зупинки чи видалення.

Питання: Для чого потрібний health endpoint?

Відповідь: Health endpoint дозволяє моніторингу перевіряти стан сервісу через HTTP. Простий status=ok може доводити лише liveness; readiness потребує перевірки потрібних залежностей, наприклад доступності БД. HTTP 200 не доводить працездатність усіх операцій API.

Питання: Що таке CI?
Відповідь: Continuous Integration (CI, неперервна інтеграція) - це практика розробки, за якої код розробників регулярно інтегрується у спільну гілку репозиторію з обов'язковим автоматичним запуском лінтерів, типізації та тестів. Це дозволяє оперативно знаходити дефекти на ранніх етапах розробки.

Питання: Що таке CD?
Відповідь: Continuous Delivery або Continuous Deployment (CD, неперервна доставка/розгортання) - це практика автоматичної підготовки готових релізних артефактів або їх автоматичного розгортання в цільове середовище після успішного проходження всіх етапів CI.

Питання: Що таке GitHub Actions?
Відповідь: GitHub Actions - це платформа автоматизації робочих процесів та CI/CD від GitHub, що дозволяє запускати сценарії тестування, збирання та розгортання безпосередньо у відповідь на події репозиторію.

Питання: Що таке workflow?
Відповідь: Workflow у GitHub Actions - це автоматизована процедура, описана у вигляді YAML-файлу в папці .github/workflows/. Вона запускається за певними подіями репозиторію (push, pull_request) і складається з однієї або кількох робіт (jobs).

Питання: Що таке job?
Відповідь: Job - це набір послідовних steps на одному runner. Jobs можуть виконуватися паралельно, якщо немає залежностей needs та вистачає ресурсів; для GitHub-hosted runner зазвичай надається окреме свіже середовище, а ізоляція self-hosted залежить від його налаштувань.

Питання: Що таке step?
Відповідь: Step (крок) - це окрема команда або дія всередині job, яка виконується послідовно на тому самому раннері. Крок може викликати готову дію (action) або виконувати shell-скрипт.

Питання: Що таке runner?
Відповідь: Runner (раннер) - це віртуальна машина або сервер під керуванням спеціального сервісу GitHub, який завантажує workflow, виконує його кроки та повертає статус і логи виконання.

Питання: Коли повинен запускатися CI?
Відповідь: CI повинен запускатися автоматично при кожному push у головні гілки репозиторію та при створенні або оновленні pull request. Це гарантує перевірку кожного внесення змін перед злиттям коду в основну кодову базу.

Питання: Для чого потрібний lint?
Відповідь: Лінтер здійснює статичний аналіз вихідного коду без його запуску з метою знаходження синтаксичних помилок, невикористаних змінних та порушень стилю коду. Це підвищує читабельність, уніфікує стиль команди та запобігає типовим програмним помилкам.

Питання: Для чого використовується Ruff?
Відповідь: Ruff перевіряє Python-код на типові помилки та порушення обраних правил через ruff check. Окремий ruff format форматує код, а ruff format --check тільки перевіряє форматування; набір lint-правил задається конфігурацією.

Питання: Для чого потрібний mypy?
Відповідь: Mypy - це статичний аналізатор типів для Python, який перевіряє код на відповідність анотаціям типів (type hints). Він допомагає виявити помилки несумісності типів і некоректні виклики функцій ще до запуску програми.

Питання: Що таке type checking?
Відповідь: Type checking перевіряє узгодженість типів аргументів, змінних і результатів із анотаціями. Статична перевірка знаходить частину помилок без запуску коду, але не гарантує відсутність runtime-помилок, особливо за наявності Any, динамічного коду або неперевірених зовнішніх даних.

Питання: Навіщо tests запускати в CI?
Відповідь: Автоматичний запуск тестів у CI гарантує, що новий код не ламає існуючу функціональність і працює коректно в чистому незалежному середовищі. Це захищає проєкт від регресійних помилок перед його злиттям або релізом.

Питання: Навіщо package build виконувати в CI?
Відповідь: Package build у CI перевіряє, що wheel і sdist збираються з цього commit у середовищі runner. Сам build не доводить повноту файлів та залежностей: потрібні перевірка вмісту, чисте встановлення wheel і запуск застосунку поза checkout.

Питання: Навіщо Docker build виконувати у CI?
Відповідь: Docker build у CI показує, що образ успішно збирається з цього commit на runner. Це не замінює запуск контейнера: окремо треба перевірити API, healthcheck, environment, права запису, persistence і завершення процесу.

Питання: Що таке GitHub Secret?
Відповідь: GitHub Secret - це захищене значення на рівні repository, environment або organization, доступне дозволеним workflow. GitHub маскує відомі значення в логах, але перетворені чи розбиті secrets можуть не замаскуватися, тому друкувати їх не можна.

Питання: Як використовувати secret у workflow?
Відповідь: У workflow секрет передається через синтаксис виразів контексту, наприклад '${{ secrets.SECRET_NAME }}', зазвичай у блоці змінних середовища env або параметрах кроку. Під час виконання раннер підставляє його значення без збереження у вихідному коді.

Питання: Що таке documentation?
Відповідь: Documentation (документація) - це структурований опис архітектури, призначення, встановлення, налаштування та використання системи для розробників і користувачів. Вона забезпечує швидке входження в проєкт і спрощує його довгострокову підтримку.

Питання: Що повинен містити README?
Відповідь: README повинен містити опис проєкту та його можливостей, вимоги до середовища, покрокові інструкції зі встановлення та локального або Docker-запуску, опис змінних конфігурації, документацію API та команди тестування.

Питання: Для чого потрібні docstrings?
Відповідь: Docstrings (рядки документації) пояснюють призначення модулів, функцій і класів безпосередньо у вихідному коді, описуючи вхідні параметри, тип поверненого значення та можливі винятки. Вони доступні через вбудовану функцію help() та використовуються генераторами документації.

Питання: Що таке OpenAPI?

Відповідь: OpenAPI - це стандартизована відкрита специфікація машиночитаного опису інтерфейсів REST API, яка формалізує всі кінцеві точки, методи, структури запитів/відповідей та коди статусів у форматі JSON або YAML.

Питання: Що таке /docs у FastAPI?

Відповідь: Маршрут /docs у FastAPI надає інтерактивну вебінтерфейсну документацію на основі Swagger UI, яка автоматично генерується за схемою OpenAPI. Вона дозволяє наочно досліджувати ендпоінти та тестувати API-запити прямо у веб-браузері.

Питання: Що таке /openapi.json?

Відповідь: Маршрут /openapi.json повертає повну схему REST API застосунку у стандартному форматі JSON згідно зі специфікацією OpenAPI. Цей документ слугує джерелом даних для генераторів документації (/docs, /redoc) та автогенерації клієнтських бібліотек.

Питання: Що таке logging?

Відповідь: Logging (логування) - це систематичний запис інформаційних повідомлень, попереджень та помилок про хід роботи програми у файл або стандартний потік виводу. Логи дозволяють відстежувати поведінку системи та діагностувати несправності у продакшні.

Питання: Чому logging краще за print?

Відповідь: Модуль logging підтримує рівні важливості (DEBUG, INFO, WARNING, ERROR), автоматично додає часові мітки та ім'я модуля, а також дозволяє маршрутизувати вивід у файли чи зовнішні сервіси. Рівень деталізації логів можна змінювати через конфігурацію без редагування вихідного коду програми.

Питання: Що таке production configuration?

Відповідь: Production configuration - це конфігурація застосунку, орієнтована на стабільність, продуктивність і сувору безпеку в робочому середовищі. Вона вимикає налагоджувальні режими (debug), використовує постійну базу даних, суворе логування та надійні секрети, передані через змінні оточення.

Питання: Чим development відрізняється від production?

Відповідь: Development-середовище оптимізоване для зручності розробки з детальними трасуваннями помилок, гарячим перезавантаженням коду та тестовими БД. Production орієнтований на максимальну продуктивність, безпеку, безперервну доступність і захист від витоку внутрішніх даних.

Питання: Що таке Docker healthcheck?

Відповідь: Інструкція HEALTHCHECK у Dockerfile задає періодичну команду перевірки того, чи застосунок у контейнері коректно функціонує (наприклад, виконує запит до /health). За її результатом Docker автоматично встановлює стан контейнера як healthy або unhealthy.

Питання: Для чого потрібний non-root user?

Відповідь: Non-root user зменшує права процесу в контейнері та наслідки компрометації застосунку. Це принцип найменших привілеїв, а не гарантія захисту хоста: залишаються ризики ядра, capabilities, неправильних mounts і доступу до Docker socket. Для БД треба окремо забезпечити права запису у volume.

Питання: Що таке multi-stage build?

Відповідь: Multi-stage build - це метод оптимізації Dockerfile з використанням кількох інструкцій FROM, де на перших етапах компілюються артефакти та збираються залежності, а

у фінальний мінімальний образ копіюються лише готові результати. Це значно скорочує розмір кінцевого образу та прибирає важкі build-залежності.

Питання: Що таке coverage?

Відповідь: Code coverage (покриття коду) - це метрика, яка відображає відсоток рядків, гілок або інструкцій вихідного коду програми, які були виконані під час прогону тестового набору. Вона вказує на наявність неперевірених фрагментів логіки в кодовій базі.

Питання: Навіщо встановлювати coverage threshold?

Відповідь: Coverage threshold, наприклад --cov-fail-under=80, робить перевірку неуспішною, якщо виміряне покриття нижче межі. Треба вказувати, чи це statement, branch або combined coverage. Поріг не доводить якість assertions і допускає непокритий код, поки загальна метрика залишається достатньою.

Питання: Що означає green CI pipeline?

Відповідь: Green CI pipeline означає успішний результат конкретного workflow run для конкретного commit. Перед release треба перевірити head SHA, усі потрібні jobs і steps, щоб обов'язкові перевірки справді виконалися, а не були skipped чи замасковані continue-on-error. Зелений статус не доводить відсутність усіх дефектів.

Висновок:

У ході роботи підготовлено повноцінне релізне середовище для системи обліку студентів: налаштовано конфігурацію контейнера Dockerfile та .dockerignore, описано змінні оточення у .env.example, розроблено пайплайн GitHub Actions для автоматичної перевірки лінтерами, типізацією mypy та повним pytest suite. Перевірено коректність розгортання та постійного збереження даних.